```
alert(eval('document.body.inne' + 'rHTML'));
```

The next problem was MySpace filtering the word "javascript", so the author used "java<NEWLINE>script" to get around this:

```
<div id="mycode" expr="alert('hah!')"
style="background:url('java↵ script:eval(document.all.mycode.expr)')">
```

Another problem for the worm's author were CSRF security tokens. Without them he couldn't send a friend request over POST. He got around it by sending a GET to the page right before adding a user and parsing the result for the CSRF token.

In the end, he got a 4 KB worm, which he injected into his profile page.

The moz-binding CSS property proved to be another way to introduce JavaScript in CSS in Gecko-based browsers (Firefox, for example).

7.4.1 Countermeasures

This example, again, showed that a blacklist filter is never complete. However, as custom CSS in web applications is a quite rare feature, it may be hard to find a good whitelist CSS filter. *If you want to allow custom colors or images, you can allow the user to choose them and build the CSS in the web application*. Use Rails' `sanitize()` method as a model for a whitelist CSS filter, if you really need one.

7.5 Textile Injection

If you want to provide text formatting other than HTML (due to security), use a mark-up language which is converted to HTML on the server-side. RedCloth is such a language for Ruby, but without precautions, it is also vulnerable to XSS.
For example, RedCloth translates `_test_` to test, which makes the text italic. However, up to the current version 3.0.4, it is still vulnerable to XSS. Get the all-new version 4 that removed serious bugs. However, even that version has some security bugs, so the countermeasures still apply. Here is an example for version 3.0.4:

```
RedCloth.new('<script>alert(1)</script>').to_html
# => "<script>alert(1)</script>"
```

Use the :filter_html option to remove HTML which was not created by the Textile processor.

```
RedCloth.new('<script>alert(1)</script>', [:filter_html]).to_html
# => "alert(1)"
```

However, this does not filter all HTML, a few tags will be left (by design), for example <a>:

```
RedCloth.new("<a href='javascript:alert(1)'>hello</a>",
[:filter_html]).to_html
# => "<p><a href="javascript:alert(1)">hello</a></p>"
```

7.5.1 Countermeasures

It is recommended to *use RedCloth in combination with a whitelist input filter*, as described in the countermeasures against XSS section.

7.6 Ajax Injection